

# LLMSVERIFIER\_BRIDGE

## LLMsVerifier Bridge — Architecture, llama.cpp Removal Map & Phased Plan

**Revision:** 1 **Last modified:** 2026-06-15T00:00:00Z **Status:** DESIGN (uncommitted) — no source edited; survey is FACT-based and cited **Authority:** Operator mandate 2026-06-15 (verbatim intent quoted below) **Scope:** Replace all llama.cpp local inference with LLMsVerifier-obtained strongest verified models across every HelixTranslate component, AND expose those models directly to a Claude Code agent session. Out-of-the-box, fully documented, full test coverage.

**Mandate (verbatim intent):** "Do NOT use any local running models using llama.cpp — only LLMsVerifier-obtained STRONGEST models. Create a BRIDGE between the HelixTranslate System and all its components AND this Claude Code session for DIRECT access to model capabilities (via the LLMsVerifier-obtained strongest models). Document everything to the smallest detail, make it work OUT OF THE BOX (no manual tuning), and FULL test coverage."

---

### 0. TL;DR for the conductor

- **llama.cpp non-test source sites to remove/redirect: 19** (enumerated §2). Plus a `_test.go` cluster in `pkg/translator/llm/` (4 files) + scattered SSH-remote-llama.cpp paths.
- **LLMsVerifier integration maturity: HIGH but with one load-bearing gap.** Both a remote-HTTP client AND a complete in-process verification pipeline already exist and are wired into unified-translator via VerifiedFactory. The gap is the **provider→base\_url materialization** when turning a verified model into a translating LLMClient (§3.3) and the **lack of a strongest-only selection contract + an agent-direct surface** (§4, §5).
- **Recommended bridge approach: a single pkg/bridge package** that wraps the existing VerifiedFactory + selection.Engine behind one Bridge facade with: (a) `BestTranslator(task)` for every component, (b) `Invoke(ctx, prompt)` for the agent-direct path, (c) an out-of-the-box bootstrap that prefers the **in-process pipeline → local SQLite store** (no server needed) and falls back to a running LLMsVerifier HTTP service if `LLMSVERIFIER_API_URL` is set. New thin binary `cmd/model-bridge` exposes

(b) to the Claude Code agent as a CLI subcommand + an optional stdio MCP server.

- **Out-of-the-box bootstrap mechanism:** `verify-models` in-process pipeline (already exists, `cmd/verify-models`) seeded from `ProvidersFromEnv()` (already exists) → SQLite store → Bridge. Zero running service required; zero manual tuning. Honest fallback to HTTP service documented.
  - **Operator-decision points (surface via `AskUserQuestion`):** see §9 — (D1) `keep-vs-remove llama.cpp` under §11.4.122, (D2) `SSH-remote-llama.cpp` worker removal, (D3) agent-direct transport choice (CLI subcommand vs full MCP server vs both), (D4) "strongest" definition (top-1 strict vs top-N fallback chain), (D5) whether the in-process pipeline or an HTTP service is the canonical OOTB source.
- 

## 1. Survey — current state (FACT, cited)

### 1.1 The LLMClient seam (the integration point)

`pkg/translator/llm/llm.go`:

- LLMClient interface (`llm.go:211–214`): `Translate(ctx, text, prompt) (string, error) + GetProviderName() string`. This is the single seam every translating client implements.
- `NewLLMTranslatorWithConfig(config)` factory (`llm.go:222–327`): a switch provider over `Provider` constants building one of ~30 per-provider clients. `ProviderLlamaCpp` constant at `llm.go:35`; its `ValidModels` row at `llm.go:71`; its switch branch case `ProviderLlamaCpp: client, err = NewLlamaCppClient(config)` at `llm.go:269–270`; the "custom model" warning path at `llm.go:246–248`.
- `TranslationConfig` carries `Provider`, `Model`, `APIKey`, `BaseURL`, `Temperature`, `MaxTokens`, `Timeout`, `Options` (aliased from `pkg/translator, llm.go:17`).

### 1.2 llama.cpp implementation (to remove)

- `pkg/translator/llm/llamacpp.go` — `LlamaCppClient` (hardware detection, model download, `llama-cli` subprocess exec). `findLlamaCppExecutable()` (`llamacpp.go:133–150`) probes `llama-cli` in `PATH/homebrew`. `Translate` shells out via `exec.CommandContext` (`llamacpp.go:158–208`). Depends on `pkg/hardware` + `pkg/models`.
- `pkg/translator/llm/llamacpp_provider.go` — `llamacpp-multi` coordinator variant (provider name asserted in `llamacpp_provider_test.go:424–428`).
- Tests: `llamacpp_test.go`, `llamacpp_provider_test.go`, `llamacpp_buildargs_defect_test.go`, plus references in `llm_test.go`, `provider_clients_test.go`, `integration_test.go`.

### 1.3 LLMsVerifier integration — what EXISTS

LLMsVerifier is already integrated **two ways**:

**(A) Remote HTTP client** — internal/verifier/client.go:

- verifier.Client hits <APIURL>/api/models (client.go:96) + /api/health (client.go:51), Bearer auth, 1-line cache TTL.
- Default APIURL = "http://localhost:8080" (config.go:50). **Requires a running LLMsVerifier server.**
- Model type aliased from the submodule's digital.vasic.llmsverifier/pkg/api (client.go:12,27). The submodule package resolves via go.mod replace digital.vasic.llmsverifier => ./llms\_verifier/llm-verifier (go.mod:82) — the canonical api.Model is at llms\_verifier/llm-verifier/pkg/api/types.go. **NOTE (§11.4.28(C) finding):** the consumed package is nested at llms\_verifier/llm-verifier/..., not at the submodule root — an honest layout observation, not a blocker for this work.

**(B) In-process verification pipeline (NO server needed)** — internal/verifier/

{run.go, pipeline.go, providers\_env.go, registry.go, persistence/sqlite\_store.go} + cmd/verify-models/main.go:

- ProvidersFromEnv() (providers\_env.go:52–67) builds OpenAI-compatible ProviderConfigs for every \*\_API\_KEY env var present (16 providers, providers\_env.go:23–41). Keys read in-memory only, never logged (providers\_env.go:48–51).
- RunVerification(...) (run.go:64–152) discovers each provider's live / models, runs the 8-step Pipeline.Verify (pipeline.go:48–92: reachability/auth/model-existence/response-format/latency/capabilities/rate-limits/error-handling) against REAL provider APIs, scores, and persists passing models to a ModelSink (SQLite).
- cmd/verify-models/main.go is the runnable OOTB entry: source api\_keys.sh && go run ./cmd/verify-models -db ./data/verified\_models.db → persists verified models. **This path needs no LLMsVerifier server at all.**

**Scoring/selection/adaptor (shared by both modes):**

- internal/verifier/scoring/engine.go — 5-component weighted scoring engine.
- internal/verifier/selection/engine.go — SelectModel(task) (engine.go:44–75) filters verified (registry.FilterVerified(MinScoreThreshold)), scores per task, returns highest; SelectFallback + buildFallbackChain (engine.go:78–120) produce a deterministic score-descending fallback chain.
- internal/services/llmsverifier\_score\_adapter.go — GetPreferences(ctx) (:87–141) returns verified models sorted score-descending with FallbackOrder rank; enforces CONST-034 (VerificationStatus=="verified" && CanSeeCode && AffirmativeResponse, :96).
- pkg/translator/llm/verified\_factory.go — **VerifiedFactory** already bridges verified-model → LLMTranslator: CreateTranslator/CreateTranslatorWithFallback (verified\_factory.go:82–140) call selector.SelectModel, build TranslationConfig{Provider, Model, APIKey: resolveAPIKey(providerID)}, and call NewLLMTranslatorWithConfig. SetKeyResolver/SetClient injection points exist.

- pkg/api/handler.go /api/v1/verified-models endpoint + createTranslator (handler.go:722–762).

**Already wired into a component:** cmd/unified-translator/main.go executeVerifiedTranslation (:528–559) + resolveProviderAPIKey env-map (:561–585) — when VerifierEnabled, it uses VerifiedFactory + verifier.NewClient instead of the legacy direct path.

## 1.4 Credentials (§11.4.10)

API keys come from env vars only (ProvidersFromEnv providers\_env.go:55, resolveProviderAPIKey unified-translator/main.go:582). Never logged. The bridge **MUST** preserve this — keys in-memory, never printed, never persisted to the SQLite store (the store holds only verified-model metadata, not keys).

## 2. llama.cpp removal / redirect map (every site)

**19 non-test source sites** + test cluster. For each: action under the bridge.

| # | Site (file:symbol)                      | Current role                                                                                                                     | Action                                                                        |
|---|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 1 | pkg/translator/llm/llamacpp.go          | LlamaCppClient impl                                                                                                              | <b>DELETE</b><br>(§11.4.124 git-history investigation first; separate commit) |
| 2 | pkg/translator/llm/llamacpp_provider.go | llamacpp-multi coordinator                                                                                                       | <b>DELETE</b> (same)                                                          |
| 3 | pkg/translator/llm/llm.go:35            | ProviderLlamaCpp const                                                                                                           | <b>REMOVE</b> const                                                           |
| 4 | pkg/translator/llm/llm.go:71            | ValidModels[ProviderLlamaCpp]                                                                                                    | <b>REMOVE</b> row                                                             |
| 5 | pkg/translator/llm/llm.go:246–248       | Ollama/LlamaCpp custom-model warning                                                                                             | <b>REMOVE</b><br>LlamaCpp arm (Ollama decision = D-ollama, §9)                |
| 6 | pkg/translator/llm/llm.go:269–270       | case ProviderLlamaCpp: factory branch                                                                                            | <b>REMOVE</b> branch; unknown/llamacpp → error                                |
| 7 | cmd/unified-translator/main.go          | -llama-binary/-llama-model/-context-size flags, provider=llamacpp, executeLlamaCppTranslation (:446–481), case "llamacpp" (:306) | <b>REMOVE</b><br>flags+branch; redirect to verified path (§4)                 |
| 8 | cmd/translate-ssh/main.go               | SSH worker runs remote llama.cpp                                                                                                 | <b>D2 operator decision</b> (§9): remove or redirect                          |

| #  | Site (file:symbol)                                                          | Current role                                            | Action                                                            |
|----|-----------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------------|
|    |                                                                             |                                                         | worker to verified API                                            |
| 9  | cmd/ssh-translation/main.go                                                 | SSH→remote llama.cpp orchestration                      | D2                                                                |
| 10 | cmd/ebook-translator/ebook_translator.go                                    | llamacpp provider option                                | <b>REMOVE</b> option; redirect to verified                        |
| 11 | cmd/markdown-translator/main.go                                             | llamacpp provider option                                | <b>REMOVE</b> option; redirect to verified                        |
| 12 | cmd/cli/main.go                                                             | llamacpp provider option                                | <b>REMOVE</b> option; redirect to verified                        |
| 13 | cmd/translator/main.go                                                      | llamacpp provider option                                | <b>REMOVE</b> option; redirect to verified                        |
| 14 | pkg/api/handler.go:573,1814                                                 | "name":"llamacpp" in models list + validProviders array | <b>REMOVE</b> from both lists; createTranslator routes via bridge |
| 15 | pkg/api/batch_handlers.go                                                   | llamacpp reference                                      | <b>REMOVE/redirect</b>                                            |
| 16 | pkg/grpc/core_translator.go, pkg/grpc/server.go                             | llamacpp provider handling                              | <b>REMOVE/redirect</b> to bridge                                  |
| 17 | pkg/grpc/proto/translator.pb.go                                             | generated — may enumerate provider                      | <b>REGENERATE</b> from .proto after enum edit (do not hand-edit)  |
| 18 | pkg/distributed/coordinator.go, pkg/distributed/pairing.go                  | distributed worker llama.cpp                            | <b>D2 decision</b> (distributed mode = remote llama.cpp workers)  |
| 19 | pkg/challenge_runner/provider_challenges.go, pkg/report/report_generator.go | llamacpp in challenge/report provider sets              | <b>REMOVE</b> llamacpp entries; add verified-bridge challenge     |

**Test cluster (remove/rewrite under TDD §11.4.43):** llama\_test.go, llama\_test\_provider\_test.go, llama\_test\_buildargs\_defect\_test.go, + llama\_test\_arms in llama\_test.go, provider\_clients\_test.go, integration\_test.go, cmd/unified-translator/ssh\_command\_test.go, cmd/cli/main\_test.go, cmd/markdown-translator/\*\_test.go.

**Out-of-scope (do NOT touch — third-party / other submodules):**

llms\_verifier/internal/benchmark/http\_provider.go, all helix\_qa/\*\*, vision\_engine/\*\*, root demo-go / test.go / tools/\*\* scrap scripts (already non-authoritative per project CLAUDE.md).

**Dead-code investigation (§11.4.124):** pkg/models (model registry/downloader) and pkg/hardware are imported ONLY by llama\_test.go for non-test source. After removal, run git log -S pickaxe on each before deleting; if they have hidden refs, wire-or-keep, do not delete on sight.

---

## 3. The gap that blocks "verified model → translating client"

### 3.1 What the verified Model carries

`api.Model` (`llms_verifier/llm-verifier/pkg/api/types.go:9–24`): `ID`, `ProviderID`, `Name`, `VerificationStatus`, `CanSeeCode`, `AffirmativeResponse`, `OverallScore`, four component scores, `Capabilities`, `Pricing`, `LastVerifiedAt`. It carries **NO API key and NO base\_url**.

### 3.2 What NewLLMTranslatorWithConfig needs

The factory switch keys on `Provider` (a known string like "openai", "deepseek") and each per-provider client supplies its own hardcoded `base_url` unless `config.BaseURL` overrides. So building a client needs: a `Provider` string the switch recognizes + an `APIKey` + (optionally) a `BaseURL`.

### 3.3 The two-part gap

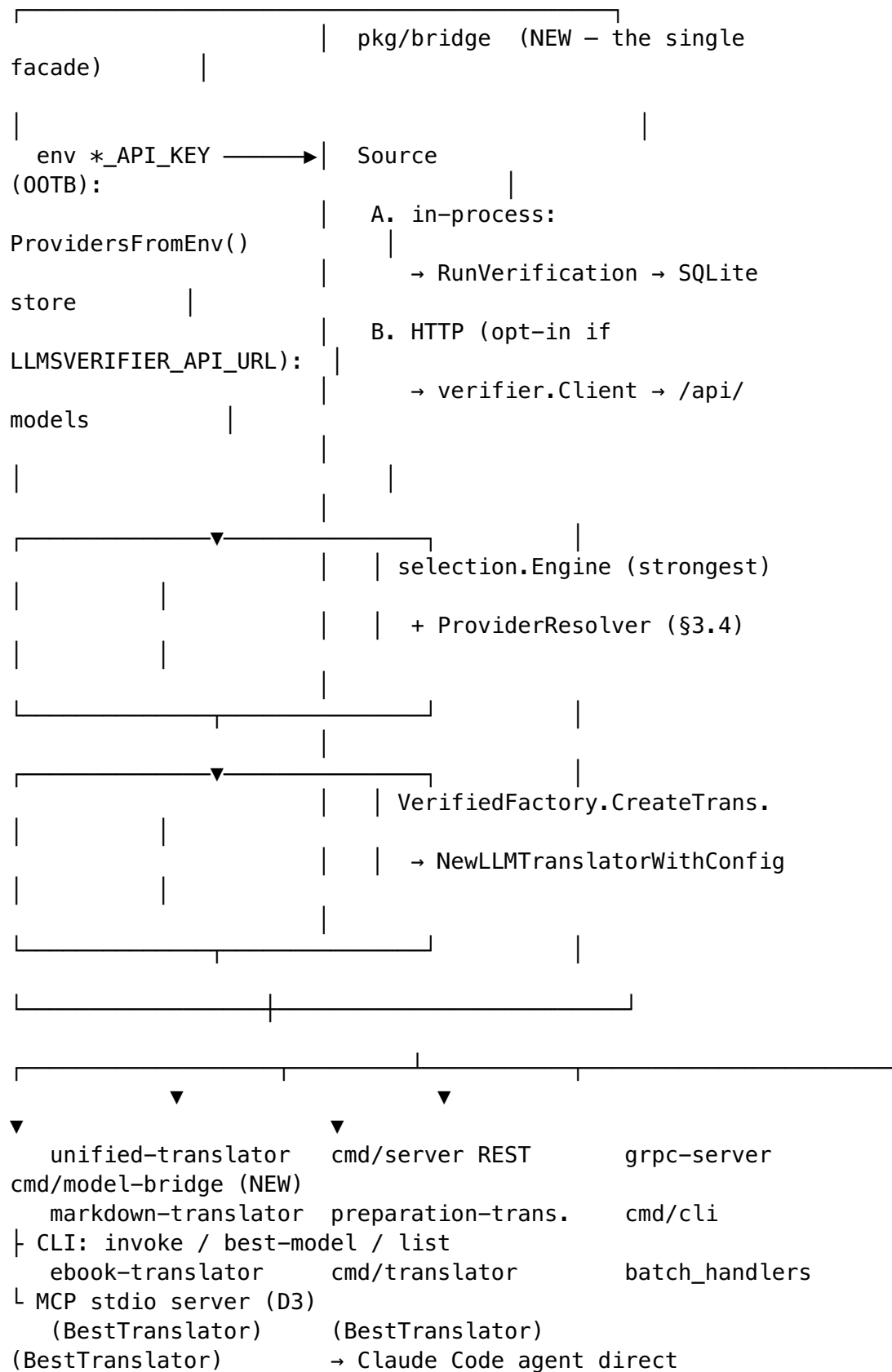
1. **Provider-ID → factory Provider mapping.** A verified model's `ProviderID` from the in-process pipeline is the `ProvidersFromEnv` id (e.g. "deepseek", "groq", "openrouter", "together", "fireworks", "nvidia", "sambanova", "hyperbolic", "novita", "siliconflow", "upstage", "cerebras", "mistral", "openai", "zhipu", "gemini"). The `NewLLMTranslatorWithConfig` switch knows only a subset (no openrouter/together/fireworks/nvidia cases). **From the HTTP server path, ProviderID is a numeric string** (`client.go:177` `fmt.Sprintf("%d", sm.ProviderID)`) — NOT a factory-recognized provider name at all. This is the load-bearing bug: a verified model selected via the HTTP path cannot currently be turned into a working client because its `ProviderID` is a number.
2. **base\_url propagation.** `envProviderSpecs` (`providers_env.go:23–41`) already holds the canonical OpenAI-compatible `base_url` per provider. The bridge must thread that `base_url` into `TranslationConfig.BaseURL` so OpenAI-compatible providers the factory does not natively case can still be served through a generic OpenAI-compatible client.

### 3.4 Resolution (design)

Introduce a **ProviderResolver** in the bridge that maps `ProviderID` → (`factoryProvider`, `baseURL`, `envKeyVar`) using `envProviderSpecs` as the single source of truth (extend it with the factory-provider name + a generic openai-compatible fallback for providers the switch does not natively case). The bridge sets `TranslationConfig{Provider, Model, APIKey, BaseURL}` so EVERY verified provider materializes — closing both halves of §3.3.

---

## 4. Bridge architecture



## 4.1 Public API (proposed)

**package** bridge

**type** Bridge **struct**

```
{ /* wraps VerifiedFactory + selection.Engine +  
   ProviderResolver */ }
```

```
// Bootstrap chooses the OOTB source (in-process pipeline →  
   SQLite) unless  
// LLMSVERIFIER_API_URL is set (then HTTP). Never requires manual  
   tuning.
```

**func** Open(ctx context.Context, opts Options) (\*Bridge, error)

```
// (a) component path – every translating component calls this:
```

**func** (b \*Bridge) BestTranslator(ctx context.Context, task  
 selection.TaskRequirements) (translator.Translator,  
 []string /\*fallbacks\*/, error)

```
// (b) agent-direct path – raw capability access for the Claude  
   Code session:
```

**func** (b \*Bridge) Invoke(ctx context.Context, system, prompt  
 string) (string, error)

**func** (b \*Bridge) BestModel(ctx context.Context, task  
 selection.TaskRequirements) (verifier.Model, error)

**func** (b \*Bridge) ListVerified(ctx context.Context)  
 ([]services.ProviderPreference, error)

BestTranslator reuses VerifiedFactory.CreateTranslatorWithFallback  
verbatim — no reimplementation (§11.4.74). Invoke builds the strongest  
LLMClient once and calls Translate(ctx, prompt, system) (the generic chat  
seam) — exposing the model's raw capability, not just translation.

## 4.2 "Strongest" contract (D4)

selection.Engine.SelectModel already returns the highest score-ranked verified  
candidate; GetPreferences already sorts score-descending with FallbackOrder.  
"Strongest" = FallbackOrder==1. The fallback chain (D4 decision: strict top-1 vs  
degrade) is already built deterministically by buildFallbackChain (selection/  
engine.go:111–120).

---

## 5. Agent-direct access (the Claude Code bridge)

cmd/model-bridge (NEW thin binary) exposes the strongest verified model to this  
agent session:

- **CLI subcommand (always):** model-bridge invoke --prompt "... " →  
prints the strongest verified model's completion; model-bridge best-model  
→ prints provider/model/score; model-bridge list → verified models JSON.  
The agent calls these via Bash. Out-of-the-box (reads env keys, runs in-process  
verify, invokes). No secrets printed.



- **MCP stdio server (D3, optional):** `model-bridge mcp` exposes tools `verified_invoke`, `verified_best_model`, `verified_list` over stdio MCP, wired into `.mcp.json` so the agent calls them as native tools (mirrors the §11.4.78 CodeGraph MCP pattern). Reuse `mcp-server-dev` skill; do NOT reimplement an MCP framework — check the catalogue (§11.4.74) for an existing Go MCP helper first.

This is the literal "bridge between ... this Claude Code session for DIRECT access to model capabilities."

---

## 6. Out-of-the-box bootstrap (no manual tuning)

`bridge.Open` resolution order (honest, §11.4.6 — no guessing about a running service):

1. If `LLMSVERIFIER_API_URL` set → `HTTP verifier.Client` mode (server is operator-provided).
2. Else → **in-process mode (canonical OOTB):** `ProvidersFromEnv()` → if a fresh SQLite store at `./data/verified_models.db` exists and is recent, load it; else run `RunVerification` once (bounded `-max/timeout`) to populate it, then select. No server, no manual config.
3. If `NO *_API_KEY` present → honest hard error listing which env vars to set (mirrors `verify-models/main.go:33-37`), never a silent fallback to `llama.cpp` (which is being removed).

§11.4.76 note: if the operator later wants `LLMsVerifier-as-a-service`, it boots on-demand via the `Containers` submodule — documented, not assumed. The in-process path means OOTB never depends on it.

---

## 7. Full test plan (§11.4.27 no-fakes-beyond-unit)

- **Unit:** `ProviderResolver` mapping table (every `ProvidersFromEnv` id → factory provider + base\_url, incl. the generic OpenAI-compatible fallback); `bridge.Open` source-selection logic; "strongest"=`FallbackOrder==1` ordering; the numeric-`ProviderID` HTTP bug regression (§3.3 part 1) as a §11.4.115 RED-then-GREEN polarity test.
- **Integration (real LLMsVerifier, real provider APIs):** in-process `RunVerification` against the providers whose keys are in env → assert a verified model is selectable AND its `BestTranslator` produces real translated text (anti-bluff §11.4 — actual target-language output, not a session id). Skip-with-reason when no keys present (§11.4.3), never a faked PASS.
- **Unforgeable-challenge E2E (§11.4.78 style):** agent calls `model-bridge invoke` / the MCP tool with a prompt whose answer is only obtainable from a live model response → assert real content. A bridge that returns canned text fails.
- **Anti-llama.cpp gate:** a pre-build gate (`CM-N0-LLAMACPP`) greps the authoritative source tree (`cmd/ + pkg/ + internal/`, excluding submodules/tests-of-removal) for `llama.cpp/llamacpp/llama-cli/ProviderLlamaCpp` and FAILs on any new occurrence; paired §1.1 mutation re-introduces a usage and asserts the gate FAILs.

- `docs/qa/` transcript per §11.4.83 for the agent-direct feature.
- 

## 8. Phased implementation plan (ordered, file-level, each phase independently buildable + testable)

### Phase 0 — Scaffolding + resolver (no removal yet; additive, builds green).

- Add `pkg/bridge/resolver.go` (ProviderResolver + mapping table extending `envProviderSpecs` with `factory-provider` + `base_url` + generic OpenAI-compatible fallback).
- Add `pkg/bridge/bridge.go` (Bridge facade, `Open`, `BestTranslator`, `BestModel`, `ListVerified`, `Invoke`) wrapping existing `VerifiedFactory`.
- Unit tests for resolver + Open source-selection. Buildable + testable alone.

### Phase 1 — Close the numeric-ProviderID gap (§3.3 part 1).

- Fix `internal/verifier/client.go:177` so HTTP-path ProviderID carries the provider name (not the numeric id), or have the resolver map server provider numbers via `/api/models` provider field. §11.4.115 RED test on the bug first.

### Phase 2 — Agent-direct surface.

- Add `cmd/model-bridge/main.go` (`invoke/best-model/list` subcommands). Unforgeable-challenge E2E test. (D3) optionally add `mcp` subcommand + `.mcp.json` wiring.

### Phase 3 — Redirect components to the bridge (additive default, `llama.cpp` still present).

- Route unified-translator, `cmd/server/pkg/api/handler.go` `createTranslator`, `grpc-server`, `preparation-translator`, `markdown-translator`, `ebook-translator`, `cmd/cli`, `cmd/translator`, `batch_handlers` through `bridge.BestTranslator`. Make `verified-bridge` the default provisioning path. Each component independently buildable.

### Phase 4 — Remove `llama.cpp` (§11.4.124 + §11.4.122).

- AskUserQuestion for D1/D2/D-ollama BEFORE removal. Then, in separate per-area commits with git-history evidence: delete `llamacpp.go` + `llamacpp_provider.go`, remove the `const/ValidModels/switch/warning` sites (#3-6), remove flags + branches (#7,10-16), regenerate proto (#17), handle distributed/SSH per D2 (#8,9,18). Investigate `pkg/models/pkg/hardware` orphaning before deletion.

### Phase 5 — Anti-bluff gate + full retest.

- Add CM-N0-LLAMACPP gate + paired mutation. Full-suite retest (§11.4.40). Update `config.json` (`default_provider` → `verified` path), `docs` (`Documentation/AGENTS.md`, `README`, `docs/CODEGRAPH.md` exclude is unaffected), and the §11.4.131 session-resumption file.
-

## 9. Operator-decision points (surface via AskUserQuestion — §11.4.66/§11.4.122)

- **D1 — llama.cpp removal confirmation (§11.4.122 MANDATORY):** llama.cpp is an existing end-user capability (`-provider llamacpp`, offline local inference). Removing it is a removal under §11.4.122 → MUST be operator-confirmed before Phase 4. Recommended: remove (per mandate), tracked as `Obsolete reason feature-removed`.
  - **D2 — SSH-remote-llama.cpp + distributed workers:** `cmd/translate-ssh`, `cmd/ssh-translation`, `pkg/distributed/*` run llama.cpp on remote workers. Remove entirely, or redirect workers to call the verified API? (Distributed mode is a separate end-user capability.)
  - **D3 — agent-direct transport:** CLI subcommand only / full MCP stdio server / both? (Both recommended; MCP matches §11.4.78 pattern.)
  - **D4 — "strongest" definition:** strict top-1 only, or top-1 + deterministic score-descending fallback chain (already built)? Recommended: top-1 with fallback chain.
  - **D5 — canonical OOTB source:** in-process pipeline+SQLite (no server, recommended) vs require a running LLMsVerifier HTTP service vs Containers-booted service. Recommended: in-process default, HTTP opt-in via `LLMSVERIFIER_API_URL`.
  - **D-ollama (minor):** Ollama is the other "local model" provider (`ProviderOllama`). The mandate names only llama.cpp. Keep Ollama, or is "no local models" intended to cover Ollama too? Recommended: keep Ollama unless operator says otherwise (§11.4.122 — do not silently remove).
- 

## 10. Catalogue-check (§11.4.74)

- **Catalogue-Check:** extend `vasic-digital/LLMsVerifier` — reuse the existing `internal/verifier+llms_verifier` submodule `pipeline/scoring/selection`; the bridge is a thin facade, NOT a reimplement of `discovery/scoring`.
- **Catalogue-Check:** reuse `VerifiedFactory + selection.Engine + score_adapter` — already in-tree; do not duplicate.
- **Agent-direct MCP:** check catalogue for an existing Go MCP helper before writing one (`mcp-server-dev` skill available).